

Decompiling Atlas JSON with jsonToCapr

Contents

0.1	Prerequisites	1
0.2	Convert JSON to Capr code in memory	1
0.3	Write Capr code to an R script	1
0.4	Source generated code and compile	2
0.5	Optional: validate with CirceR SQL generation	2
0.6	Using skip mode for partial conversion	2
0.7	Capture skipped items programmatically	2
0.8	Round-trip pattern	3
0.9	Troubleshooting	3

This vignette shows how to convert an Atlas/Circe cohort JSON file into executable Capr R code using `jsonToCapr()` and `jsonToCaprFile()`.

The common workflow is:

1. Export a cohort definition JSON from Atlas.
2. Convert JSON to Capr code.
3. Source the generated R code.
4. Compile the resulting Capr cohort and (optionally) generate SQL with CirceR.

0.1 Prerequisites

```
library(Capr)
```

You need a cohort JSON exported from Atlas. In this vignette, we refer to it as `cohort.json`.

0.2 Convert JSON to Capr code in memory

Use `jsonToCapr()` when you want the generated code as a character string.

```
jsonPath <- "cohort.json"

caprCode <- jsonToCapr(jsonPath, mode = "strict")
cat(caprCode)
```

`mode = "strict"` stops when unsupported features are encountered. This is the safest mode when you need a faithful conversion.

0.3 Write Capr code to an R script

Use `jsonToCaprFile()` to write generated code directly to disk.

```
outRPath <- "cohort_from_atlas.R"

jsonToCaprFile(
  jsonPath = jsonPath,
  outRPath = outRPath,
```

```
mode = "strict"
)
```

The generated script includes `library(Capr)` and can be sourced directly.

0.4 Source generated code and compile

By convention, generated scripts define a `cohortDef` object.

```
source(outRPath)

# cohortDef should now exist in your environment
cohortJson <- toCohortJson(cohortDef)
```

At this point, `cohortJson` can be:

- saved and imported back into Atlas, or
- passed to CirceR to generate SQL.

0.5 Optional: validate with CirceR SQL generation

```
sql <- CirceR::buildCohortQuery(
  expression = CirceR::cohortExpressionFromJson(cohortJson),
  options = CirceR::createGenerateOptions(generateStats = FALSE)
)

nchar(sql) > 0
```

If SQL generation succeeds, the converted cohort is structurally valid for CirceR.

0.6 Using skip mode for partial conversion

`mode = "skip"` attempts to continue conversion by omitting unsupported parts and annotating them in output comments.

```
caprCodeSkip <- jsonToCapr(jsonPath, mode = "skip")
cat(caprCodeSkip)
```

This mode is useful for exploratory migration, but you should manually review and complete skipped sections before production use.

0.7 Capture skipped items programmatically

For reporting or regression checks, use `returnSkipped = TRUE`.

```
result <- jsonToCapr(
  jsonPath = jsonPath,
  mode = "skip",
  returnSkipped = TRUE
)

names(result)
# [1] "lines" "skipped" "emptyGroupWarnings"

result$skipped
```

0.8 Round-trip pattern

A practical migration pattern is:

1. `jsonToCaprFile()` from Atlas JSON.
2. `source()` generated script.
3. `toCohortJson()` to JSON.
4. `CirceR::cohortExpressionFromJson() + buildCohortQuery()` as a final validation.

This creates a reproducible, version-controlled bridge from Atlas-authored cohorts to Capr-authored pipelines.

0.9 Troubleshooting

- If strict mode fails, run with `mode = "skip"` to identify unsupported pieces.
- Review generated `# SKIPPED:` comments and complete the cohort manually in Capr.
- Ensure concept sets referenced by the generated code are present and correctly represented before final SQL generation.